Keyboard Control

Keyboard Control
1. Course Content
2. Preparation
3. Key Operations
    3.1 Chassis Motion Control
    3.2 Speed Adjustment
    3.3 Robotic Arm Control
4. Source Code Analysis

1. Course Content

- Control the robot car's chassis and gimbal camera via the keyboard
- Understand the internal implementation source code of keyboard control

2. Preparation

- Start the chassis driver node

```
ros2 launch yahboomcar_bringup car_base.launch.py
```

- Start keyboard control in the terminal

> **Tip**
>
> It is recommended to start it in an SSH-connected terminal rather than in VNC; the key experience will be smoother

```
ros2 run common_demo keyboard_control
```

```
jetson@yahboom:~$ ros2 run common_demo keyboard_control keyboard_control

Reading from the keyboard  and Publishing to Twist!
---------------------------
Moving around:
   u    i    o
   j    k    l
   m    ,    .

For Holonomic mode (strafing), hold down the shift key:
---------------------------
   U    I    O
   J    K    L
   M    <    >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds
w/x : increase/decrease only linear speed (step 0.02, range 0.01~2.0)
e/c : increase/decrease only angular speed (step 0.1, range 0.1~3.0)

Arm Control:
   1-6 : select joint 1-6
   a   : increase selected joint angle
   d   : decrease selected joint angle
   0   : reset arm to initial pose
```

3. Key Operations

> **Important**
>
> When using keyboard control, you need to switch the input method to English; otherwise the keys will not take effect.

3.1 Chassis Motion Control

Key	Function
`u`	Move forward-left (forward + turn left)
`i`	Forward
`o`	Move forward-right (forward + turn right)
`j`	Turn left in place
`k`	Stop
`l`	Turn right in place
`m`	Move backward-left (backward + turn left)

Key	Function
,	Backward
.	Move backward-right (backward + turn right)

3.2 Speed Adjustment

Key	Function
q	Increase both maximum linear and angular velocity by 10%
z	Decrease both maximum linear and angular velocity by 10%
w	Increase linear velocity only by 10%
x	Decrease linear velocity only by 10%
e	Increase angular velocity only by 10%
c	Decrease angular velocity only by 10%

3.3 Robotic Arm Control

- Keyboard control supports single-joint selection adjustment and one-key pose reset.

Key	Function Description	Corresponding Action and Output
1 ~ 6	**Select the joint currently being controlled** (Joint 1 ~ 6)	The terminal displays the selected joint number, current angle, and allowed range of motion
a / A	**Increase** the currently selected joint angle	Publish an `ArmJoint` message; automatically limits when reaching the upper limit
d / D	**Decrease** the currently selected joint angle	Publish an `ArmJoint` message; automatically limits when reaching the lower limit
0	**One-key reset** of all 6 joints to the initial pose	Restore to the preset initial pose

> **Note**
>
> To close keyboard control, simply press ctrl+c in the opened terminal to end the process and close the terminal window

4. Source Code Analysis

- Source code path

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/control/keyboard_control.py
```

- `reset_arm`: reads the `init_pose` parameter to get the robotic arm's initial angles, calls `pubSixArm` to send the reset command, and clears the terminal buffer after waiting 2 seconds
- `adjust_joint`: adjusts a single joint angle according to the joint index; it first validates the index's legality, then clamps via `arm_bounds` to prevent exceeding the limit. After publishing the `ArmJoint` command, it clears the buffer to ensure the arm stops as soon as the key is released
- `getKey`: reads a single key character unbuffered via `os.read`
- `vels`: formats and outputs the current linear and angular velocity values
- `main` main loop: checks whether stdin is a TTY and sets cbreak mode to enable character-by-character input; it loops reading keys, maps `u/i/o/j/k/l/m/,/.` to chassis motion directions, uses `q/z/w/x/e/c` to adjust speed (±10%, with upper/lower limit clamping), uses `1~6` to select a joint, `a/d` to fine-tune the joint angle, `0` to reset the robotic arm, sets the velocity to zero for other keys, and exits on `Ctrl+C`

```
    def reset_arm(self):
        """Reset arm to initial pose"""
        init_pose_raw = self.get_parameter('init_pose').value
        self.joint_positions = [int(x) for x in init_pose_raw]

        # Publish reset command directly
        self.pubSixArm(self.joint_positions, runtime=2000)
        time.sleep(2.0)
        termios.tcflush(sys.stdin.fileno(), termios.TCIFLUSH)

    def adjust_joint(self, joint_idx, delta):
        """Adjust a specific joint angle with bounds checking"""
        if joint_idx < 0 or joint_idx >= len(self.joint_positions):
            self.get_logger().warn(f"Invalid joint index: {joint_idx}")
            return

        # Apply adjustment with bounds checking
        self.joint_positions[joint_idx] = max(
            self.arm_bounds[joint_idx][0],
            min(self.arm_bounds[joint_idx][1],
                self.joint_positions[joint_idx] + delta)
        )

        # Publish arm command directly
        arm_msg = ArmJoint()
        arm_msg.id = joint_idx + 1  # IDs are 1-based
        arm_msg.joint = int(self.joint_positions[joint_idx])
        arm_msg.time = self.arm_joint_time
        self.arm_joint_pub.publish(arm_msg)
        time.sleep(self.arm_joint_time / 1000.0)
        # Discard buffered key-repeat chars so the arm stops on key release
        termios.tcflush(sys.stdin.fileno(), termios.TCIFLUSH)
```

```
def getKey():
    # Unbuffered read: leaves extra chars in the kernel buffer so tcflush can drop them
    key = os.read(sys.stdin.fileno(), 1).decode(errors='ignore')
    return key

def vels(speed, turn):
    return "currently:\tspeed %.2f\tturn %.1f " % (speed, turn)

def main(args=None):
    global settings
    if args is None:
        args = sys.argv

    # keyboard control requires a real terminal (stdin as TTY)
    if not sys.stdin.isatty():
        print("[keyboard_control] stdin is not a TTY, keyboard control needs a real
terminal.")
        print("[keyboard_control] Run it with: ros2 run common_demo keyboard_control")
        sys.exit(1)
    settings = termios.tcgetattr(sys.stdin)

    rclpy.init(args=args)
    keyboard_node = KeyBoardControlNode()

    try:
        # cbreak mode: no echo & char-by-char input, but keeps output processing (\n ->
\r\n)
        tty.setcbreak(sys.stdin.fileno())
        print(msg)
        print(vels(keyboard_node.speed, keyboard_node.turn))

        status = 0
        while rclpy.ok():
            key = getKey()
            if key in moveBindings.keys():
                keyboard_node.x = moveBindings[key][0]
                keyboard_node.y = moveBindings[key][1]
                keyboard_node.z = moveBindings[key][2]
                keyboard_node.th = moveBindings[key][3]
            elif key in speedBindings.keys():
                keyboard_node.speed = round(
                    min(SPEED_MAX, max(SPEED_MIN, keyboard_node.speed + speedBindings[key]
[0])), 2)
                keyboard_node.turn = round(
                    min(TURN_MAX, max(TURN_MIN, keyboard_node.turn + speedBindings[key]
[1])), 1)

                print(vels(keyboard_node.speed, keyboard_node.turn))
                if (status == 14):
                    print(msg)
                status = (status + 1) % 15
            elif key in ['1', '2', '3', '4', '5', '6']:
```

```
                # Select joint 1-6
                keyboard_node.selected_joint = int(key) - 1
                print(f"Selected joint{key}")
            elif key == 'a':
                # Decrease selected joint angle
                keyboard_node.adjust_joint(keyboard_node.selected_joint,
keyboard_node.arm_joint_step)
            elif key == 'd':
                # Increase selected joint angle
                keyboard_node.adjust_joint(keyboard_node.selected_joint, -
keyboard_node.arm_joint_step)
            elif key == '0':
                # Reset arm to initial pose
                keyboard_node.reset_arm()
                print(f"Arm reset to initial pose: {keyboard_node.joint_positions}")
            else:
                keyboard_node.x = 0
                keyboard_node.y = 0
                keyboard_node.z = 0
                keyboard_node.th = 0
                if (key == '\x03'):
                    break

            twist = Twist()
            twist.linear.x = keyboard_node.x * keyboard_node.speed
            twist.linear.y = keyboard_node.y * keyboard_node.speed
            twist.linear.z = keyboard_node.z * keyboard_node.speed
            twist.angular.x = 0.0
            twist.angular.y = 0.0
            twist.angular.z = keyboard_node.th * keyboard_node.turn
            keyboard_node.cmd_vel_pub.publish(twist)

    except KeyboardInterrupt:
        pass  # CTRL-C: quit gracefully
    except Exception as e:
        keyboard_node.get_logger().error(f"Error: {e}")

    finally:
        twist = Twist()
        twist.linear.x = 0.0
        twist.linear.y = 0.0
        twist.linear.z = 0.0
        twist.angular.x = 0.0
        twist.angular.y = 0.0
        twist.angular.z = 0.0
        keyboard_node.cmd_vel_pub.publish(twist)

        termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)

        keyboard_node.destroy_node()
        rclpy.shutdown()
```